

创新实验 - 工作报告

日期：2026 年 1 月 15 日

一、工作概述

今天主要完成了 LPR 算法的学习研究和电机系统的配置工作。

- 上午与同学一起深入研究了样例代码中提供的 LPR 算法和仿真代码。
- 下午主要完成了电机的零点配置、复位操作和编号设置。

二、主要工作内容

2.1 LPR 算法研究与仿真代码学习

上午跟同学一起研究了样例代码里的 LPR 算法和仿真程序。说实话一开始看代码的时候有点懵,这个算法涉及到五连杆机构的运动学计算,还要处理腿部支撑力和地面接触检测,挺复杂的。

我们主要看了 VMC(虚拟模型控制)部分的实现代码。这个算法的核心思路是通过五连杆机构来实现腿的伸缩和摆动,然后把末端的力和力矩转换成各个关节电机需要输出的扭矩。代码里定义了一个结构体来存储五连杆的各种参数:

```
void VMC_init(vmc_leg_t *vmc)//给杆长赋值
{
    vmc->l5=0.08f;//AE 长度 //单位为m
    vmc->l1=0.075f;//单位为m
    vmc->l2=0.14f;//单位为m
    vmc->l3=0.14f;//单位为m
    vmc->l4=0.075f;//单位为m
}
```

初始化函数很简单,就是把五连杆的各段长度赋值进去。这些长度参数是根据我们实际机械结构测量得到的。

接下来是核心的运动学计算部分。这个函数计算当前腿部的姿态角 θ 和腿长 L_0 ,这些量后面会用在 LQR 控制器里:

```

void VMC_calc_1_right(vmc_leg_t *vmc,INS_t *ins,float dt)//计算theta 和
d_theta 给Lqr 用,同时也计算腿长L0
{
    static float PitchR=0.0f;
    static float PithGyroR=0.0f;
    PitchR=ins->Pitch;
    PithGyroR=ins->Gyro[0];

    vmc->YD = vmc->l4*arm_sin_f32(vmc->phi4);//D 的y 坐标
    vmc->YB = vmc->l1*arm_sin_f32(vmc->phi1);//B 的y 坐标
    vmc->XD = vmc->l5 + vmc->l4*arm_cos_f32(vmc->phi4);//D 的x 坐标
    vmc->XB = vmc->l1*arm_cos_f32(vmc->phi1); //B 的x 坐标

    vmc->lBD = sqrt((vmc->XD - vmc->XB)*(vmc->XD - vmc->XB) + (vmc->YD -vmc->YB)*(vmc->YD - vmc->YB));

    vmc->A0 = 2*vmc->l2*(vmc->XD - vmc->XB);
    vmc->B0 = 2*vmc->l2*(vmc->YD - vmc->YB);
    vmc->C0 = vmc->l2*vmc->l2 + vmc->lBD*vmc->lBD - vmc->l3*vmc->l3;
    vmc->phi2 = 2*atan2f((vmc->B0 + sqrt(vmc->A0*vmc->A0 + vmc->B0*vmc->B0 - vmc->C0*vmc->C0)),vmc->A0 + vmc->C0);
    vmc->phi3 = atan2f(vmc->YB-vmc->YD+vmc->l2*arm_sin_f32(vmc->phi2),vmc->XB-vmc->XD+vmc->l2*arm_cos_f32(vmc->phi2));

    //C 点直角坐标
    vmc->XC = vmc->l1*arm_cos_f32(vmc->phi1) + vmc->l2*arm_cos_f32(vmc->phi2);
    vmc->YC = vmc->l1*arm_sin_f32(vmc->phi1) + vmc->l2*arm_sin_f32(vmc->phi2);
    //C 点极坐标
    vmc->L0 = sqrt((vmc->XC - vmc->l5/2.0f)*(vmc->XC - vmc->l5/2.0f) + vmc->YC*vmc->YC);

    vmc->phi0 = atan2f(vmc->YC,(vmc->XC - vmc->l5/2.0f));//phi0 用于计算Lqr 需要的theta
    vmc->alpha=pi/2.0f-vmc->phi0 ;

    vmc->theta=pi/2.0f-PitchR-vmc->phi0;//得到状态变量1
    vmc->d_theta=(-PithGyroR-vmc->d_phi0);//得到状态变量2

    vmc->d_L0=(vmc->L0-vmc->last_L0)/dt;//腿长L0 的一阶导数
    vmc->dd_L0=(vmc->d_L0-vmc->last_d_L0)/dt;//腿长L0 的二阶导数
}

```

这段代码是在做坐标变换和五连杆的运动学解算。先根据两个关节电机的角度 ϕ_1 和 ϕ_4 算出各个连杆节点的位置,然后用几何关系求解出其他连杆的角度。最后算出腿的等效长度 L_0 和摆角 θ ,这两个量是后面做平衡控制最关键的状态变量。

还有一个比较重要的函数是力矩映射,把我们期望的腿部支撑力 F_0 和摆动力矩 T_p 转换成两个关节电机应该输出的扭矩:

```
void VMC_calc_2(vmc_leg_t *vmc)// 计算期望的关节输出力矩
{
    vmc->j11 = (vmc->l1*arm_sin_f32(vmc->phi0-vmc->phi3)*arm_sin_f32(vmc->phi1-vmc->phi2))/arm_sin_f32(vmc->phi3-vmc->phi2);
    vmc->j12 = (vmc->l1*arm_cos_f32(vmc->phi0-vmc->phi3)*arm_sin_f32(vmc->phi1-vmc->phi2))/(vmc->L0*arm_sin_f32(vmc->phi3-vmc->phi2));
    vmc->j21 = (vmc->l4*arm_sin_f32(vmc->phi0-vmc->phi2)*arm_sin_f32(vmc->phi3-vmc->phi4))/arm_sin_f32(vmc->phi3-vmc->phi2);
    vmc->j22 = (vmc->l4*arm_cos_f32(vmc->phi0-vmc->phi2)*arm_sin_f32(vmc->phi3-vmc->phi4))/(vmc->L0*arm_sin_f32(vmc->phi3-vmc->phi2));

    vmc->torque_set[0]=vmc->j11*vmc->F0+vmc->j12*vmc->Tp;// 得到 RightFront 的输出轴期望力矩
    vmc->torque_set[1]=vmc->j21*vmc->F0+vmc->j22*vmc->Tp;// 得到 RightBack 的输出轴期望力矩
}
```

这个函数用到了雅可比矩阵的思想。 j_{11} 、 j_{12} 、 j_{21} 、 j_{22} 这几个系数组成了雅可比矩阵,通过矩阵变换把末端的力映射到关节空间。这样上层控制器只要给出期望的腿部支撑力和摆动力矩,底层就能自动算出各个电机该出多大力。

我们还特别关注了地面接触检测的代码:

```
uint8_t ground_detectionR(vmc_leg_t *vmc,INS_t *ins)
{
    vmc->FN=vmc->F0*arm_cos_f32(vmc->theta)+vmc->Tp*arm_sin_f32(vmc->theta)/vmc->L0+6.0f;

    if(vmc->FN<5.0f)
    { // 离地了
        return 1;
    }
    else
    {
        return 0;
    }
}
```

这个函数通过计算腿部对地面的法向力来判断是否接地。如果支撑力小于 5N 就认为腿已经离地了。这个判断对于跳跃或者跨越障碍物的时候很有用,可以切换不同的控制模式。

通过跑仿真程序,我们调整了一些参数,发现腿长 L_0 的变化对整个系统的稳定性影响很大。腿太短的话容易失去平衡,腿太长又会导致电机力矩不够。另外 IMU 的俯仰角 Pitch 数据必须准确,不然算出来的 θ 就是错的,整个控制就垮了。

2.2 电机系统配置与接口修改

下午主要干了两件事:给电机配置零点和编号,然后把通信接口从 Ps2 改成了 Uart。

电机零点配置

我们用的是 DM 电机,这种电机自带编码器,但是上电后它不知道自己在什么位置,所以必须先设置零点。配置的时候要先把机械结构摆到一个标准位置,比如让五连杆处于完全展开的状态,然后在上位机软件里点击“设置零点”按钮。

一共有 4 个电机要配置:左腿前关节、左腿后关节、右腿前关节、右腿后关节。每个电机配完后我都测试了一下复位功能,就是让电机回到零点位置,看看机械结构是不是回到了初始姿态。

电机编号设置

- 1 号电机:右腿前关节(ϕ_1)
- 2 号电机:右腿后关节(ϕ_4)
- 3 号电机:左腿前关节(ϕ_1)
- 4 号电机:左腿后关节(ϕ_4)

三、技术要点总结

3.1 VMC 算法和五连杆机构的理解

今天最大的收获就是弄明白了 VMC 算法是怎么工作的。以前只知道五连杆机构可以实现腿的伸缩,但具体怎么算、怎么控制一直没搞清楚。

五连杆机构的核心思想是用两个电机驱动五根连杆,通过机构的几何约束实现腿的伸缩和摆动。关键是要建立正运动学(从关节角度算腿的位置)和逆运动学(从期望

的腿部力算电机扭矩)两套方程。正运动学相对简单,就是坐标变换和三角函数计算;逆运动学用到了雅可比矩阵,需要对五连杆机构做力的分析。

还有一点体会是,IMU 的数据质量非常重要。我们在代码里看到有个 EKF(扩展卡尔曼滤波)用来融合陀螺仪和加速度计的数据:

```
typedef struct
{
    uint8_t Initialized;
    KalmanFilter_t IMU_QuaternionEKF;
    uint8_t ConvergeFlag;
    uint8_t StableFlag;

    float q[4];           // 四元数估计值
    float GyroBias[3];    // 陀螺仪零偏估计值

    float Gyro[3];
    float Accel[3];

    float Roll;
    float Pitch;
    float Yaw;

    float Q1; // 四元数更新过程噪声
    float Q2; // 陀螺仪零偏过程噪声
    float R;  // 加速度计量测噪声

    float dt; // 姿态更新周期
    float ChiSquareTestThreshold; // 卡方检验阈值
} QEKF_INS_t;
```

这个结构体定义了姿态估计需要的所有状态量。用四元数来表示姿态可以避免万向锁问题,而且计算效率比旋转矩阵高。陀螺仪零偏的在线估计也很重要,不然时间长了姿态角会漂。

LQR 控制器的参数是用多项式拟合的方法根据腿长来动态调整的:

```
float LQR_K_calc(float *coe,float len)
{
    return coe[0]*len*len*len+coe[1]*len*len+coe[2]*len+coe[3];
}
```

这个函数用三次多项式来计算 LQR 的 K 矩阵系数,len 就是当前的腿长。这样做的好处是,腿长变化的时候控制器参数能自动适应,不用手动切换不同的控制增益。